DALHOUSIE UNIVERSITY

Data Preprocessing Foundations

Tidy data, missing values, and transformation

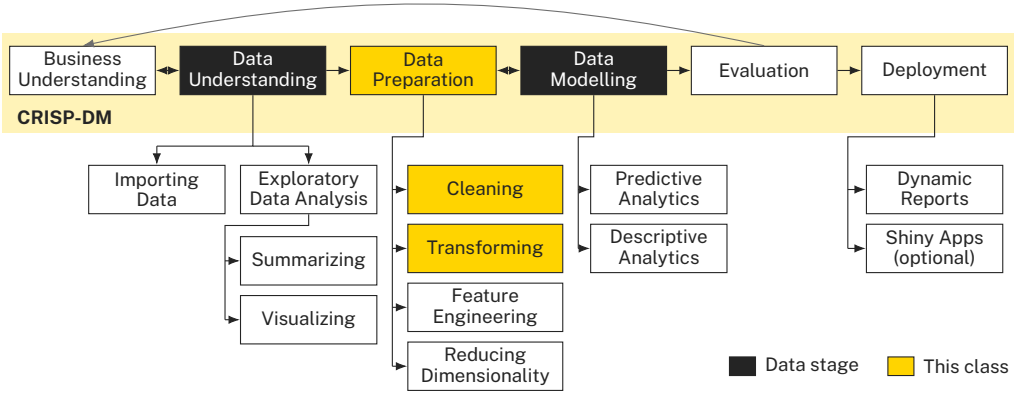
Prof. Gabriel Spadon, PhD
Faculty of Computer Science
spadon@dal.ca

Continuing into Week 2

- Lab 02 begins in Class 04 and builds on your Lab 01 workflow.
- Use R or Python, locally or in an approved cloud environment.
- Keep assessed work private until the deadline and instructor release.
- Check the course repository for current practical instructions.

Bring the record of the decisions you made in Lab 01. Lab 02 uses a new dataset, chosen in Class 04 from the Lab 02 catalogue.

Today, Data Preparation



Cleaning and transforming build on the summaries from Class 02.

Match Each Problem to an Action

Problem	Preparation step
A temperature column mixes units	Transform to a common unit.
9999 is a documented missing code	Clean the representation of missingness.
A total and a count suggest a ratio	Create a derived variable.
Too many variables for a useful analysis	Consider dimensionality reduction.

A unit conversion changes the value and the unit together, for example $C = (F - 32) \times 5/9$ for a Fahrenheit reading.

Four Preparation Categories

Cleaning	Interpret types, missing codes, and errors.
Transformation	Change units, scale, or representation.
Feature engineering	Derive useful variables from recorded information.
Dimensionality reduction	Reduce the number of variables used in an analysis.

Today focuses on cleaning and transformation. Class 04 extends these ideas to feature engineering and data reduction.

Check Meaning Before Changing a Value

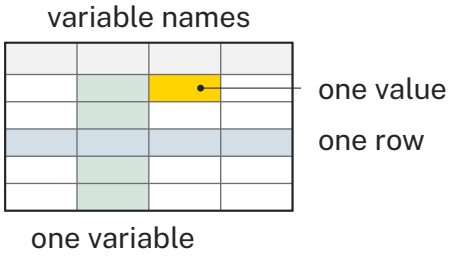
What does one row represent? Which units and types are expected? Which missing codes were documented?

A missing value can remain missing. Filling every gap is not the goal.

Start with the Observational Unit

- One variable per column.
- One observation per row.
- One value per cell.
- One type of observational unit per table.

For the next example, an observation is **one student in one course**.



Organize the Grades Table

We want one row for each student-course pair. Which columns need to change?

Think for two minutes, compare with a neighbour, then explain one change.

The example's data dictionary says grades range from 0 to 100 and 9999 means missing.

Name	Math	English	DegreeYear
Luis	90	86	CS|2020
Maria	85	9999	Math|2013
Jose	70	60	CS|2015

Three Decisions, with Different Reasons

Name	Math	English	DegreeYear
Luis	90	86	CS|2020
Maria	85	9999	Math|2013
Jose	70	60	CS|2015

1. Split `DegreeYear` into two variables.
2. Reshape course names into a `Course` column and scores into `Grade`.
3. Replace the documented missing code with `NA` or `NaN`.

The first two decisions reshape the table. The third cleans one value, as the data dictionary specifies.

One Row per Student and Course

Name	Course	Grade	Degree	Year
Luis	Math	90	CS	2020
Maria	Math	85	Math	2013
Jose	Math	70	CS	2015
Luis	English	86	CS	2020
Maria	English	*NA*	Math	2013
Jose	English	60	CS	2015

Three student rows become six student-course rows. The raw table is retained. Maria's missing English grade remains a missing value.

A student-level analysis could instead treat Math and English grades as separate variables.

Reshape the Grades

Continue from the runnable setup in the lecture notes; it creates `clean_grades`.

R

```r
tidy_grades <- tidyr::separate_wider_delim(
  clean_grades, DegreeYear, delim = "|",
  names = c("Degree", "Year")
)
tidy_grades$Year <- as.integer(tidy_grades$Year)
tidy_grades <- tidyr::pivot_longer(
  tidy_grades, c(Math, English),
  names_to = "Course", values_to = "Grade",
  cols_vary = "slowest"
)
tidy_grades <- tidy_grades[c(
  "Name", "Course", "Grade", "Degree", "Year"
)]
print(tidy_grades)
```

Python

```python
tidy_grades = clean_grades.copy()
tidy_grades[["Degree", "Year"]] = (
    tidy_grades["DegreeYear"]
    .str.split("|", expand=True, regex=False)
)
tidy_grades["Year"] = (
    tidy_grades["Year"].astype(int))
tidy_grades = tidy_grades.drop(
    columns="DegreeYear"
).melt(id_vars=["Name", "Degree", "Year"],
       var_name="Course", value_name="Grade")
tidy_grades = tidy_grades[[
    "Name", "Course", "Grade", "Degree", "Year"
]]
print(tidy_grades)
```

Explicit Date and String Rules

Normalize labels only when the data dictionary says case and surrounding spaces carry no meaning.

R

```r
labels <- c(" North ", "north", "SOUTH")
trimws(tolower(labels))
# "north" "north" "south"
as.Date(c("2026-09-15", "2026-09-16"),
        format = "%Y-%m-%d")
# Timestamps also need a time zone.
```

Python

```python
labels = pd.Series([" North ", "north", "SOUTH"])
print(labels.str.strip().str.lower().tolist())
# ['north', 'north', 'south']
pd.to_datetime(["2026-09-15", "2026-09-16"],
               format="%Y-%m-%d")
# Timestamps also need a time zone.
```

R also offers `stringr` for strings and `lubridate` for dates; Python offers string methods and timezone-aware datetime tools. Use either language.

A Gap Needs an Explanation

First record what is missing. Then ask how the gap could have arisen and what information supports that explanation.

Three Assumptions About Missingness

MCAR Missing completely at random. Missingness is independent of both observed and unobserved values.

MAR Missing at random. After accounting for observed information, missingness does not depend on the unobserved values.

MNAR Missing not at random. Missingness still depends on unobserved information after accounting for what was observed.

These describe the recording process, not whether the measurements themselves are random.

One Example of Each

	Instrument example	Survey example
MCAR	Readings are lost independently of measured values and recording order.	A random subset of forms is lost, independently of responses.
MAR	Failures depend on recorded surface type, not on the reading within a type.	Response depends on recorded delivery method, not on the answer within a method.
MNAR	An unrecorded extreme reading makes failure more likely.	Response depends on the undisclosed answer, even given recorded variables.

ACTIVITY

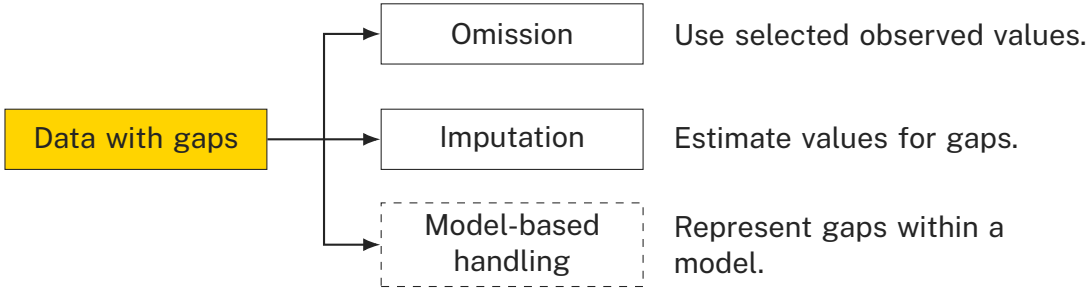
What Evidence Supports the Assumption?

A sensor fails more often outdoors than indoors. Both location and readings are recorded when available.

- What extra assumption would make this MAR?
- What if failure also depends on an unrecorded, extreme sensor reading?

We can look for observed associations, but the observed data alone cannot distinguish MAR from MNAR.

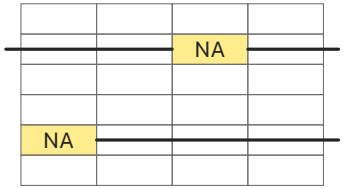
Three Ways to Proceed



Complete Cases and Available Cases

DALHOUSIE UNIVERSITY

Complete-case deletion
Use only rows observed on every variable needed for this analysis.



Two incomplete rows omitted

Available-case summaries
Use every observed value needed for each summary. A two-variable statistic uses complete pairs, hence *pairwise* deletion.

Different summaries may describe different subsets. Always report their counts.

MCAR does not add selection bias to a mean from a representative sample; fewer observations still mean less information.

Keep the Count Beside the Mean

Use only the two score columns in `clean_grades`.

R

```r
scores <- clean_grades[c("Math", "English")]
complete <- scores[complete.cases(scores), ]
nrow(complete) # 2
colMeans(complete)
# Math 80, English 73
colMeans(scores, na.rm = TRUE)
# Math 81.66667, English 73
colSums(!is.na(scores))
# Math 3, English 2
```

Python

```python
scores = clean_grades[["Math", "English"]]
complete = scores.dropna()
print(len(complete)) # 2
print(complete.mean())
# Math 80.0, English 73.0
print(scores.mean())
# Math 81.666667, English 73.000000
print(scores.notna().sum())
# Math 3, English 2
```

R needs `na.rm=TRUE`; pandas means skip missing values by default. Report three Math scores and two English scores.

Ways to Fill a Gap

Central value	A mean, median, or mode, selected for the variable type.
Ordered measurements	Previous value (LOCF), baseline value (BOCF), or interpolation between observations. Requires a justified time-series assumption.
Prediction	Other recorded variables inform a predicted numeric value or category.
Multiple imputation	Several plausible completed datasets and a combined analysis that represents uncertainty.

Imputed values are estimates. Retain a record of which entries were originally missing.

ACTIVITY

Predict the Effect of Mean Fill

Replace two gaps with the observed mean, 10.5. What happens to the mean and sample standard deviation?

Write a prediction and a reason before calculating.

Compare Mean and Median Fill

R

```r
observed <- c(2, 4, 4, 5, 7, 9, 12, 41)
with_gaps <- c(observed, NA, NA)
mean_filled <- with_gaps
mean_filled[is.na(mean_filled)] <-
  mean(observed)
median_filled <- with_gaps
median_filled[is.na(median_filled)] <-
  median(observed)
c(mean(mean_filled), sd(mean_filled))
# 10.5, 11.22497
c(mean(median_filled), sd(median_filled))
# 9.6, 11.3842
```

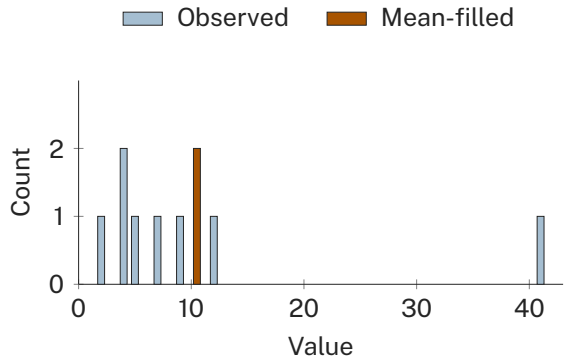
Python

```python
observed = pd.Series(
    [2, 4, 4, 5, 7, 9, 12, 41], dtype=float)
with_gaps = pd.concat(
    [observed, pd.Series([np.nan, np.nan])],
    ignore_index=True)
mean_filled = with_gaps.fillna(observed.mean())
median_filled = with_gaps.fillna(
    observed.median())
print(mean_filled.mean(), mean_filled.std())
# 10.5 11.224972160321824
print(median_filled.mean(), median_filled.std())
# 9.6 11.384199576606166
```

Both fills keep ten values. Mean fill keeps the mean at 10.5; median fill moves it to 9.6.

The Mean Stays, the Spread Shrinks

- Preserves the observed mean exactly.
- Reduces the sample variance by adding zero deviations.
- Can distort relationships between variables.
- Does not remove selection bias in the observed values.



Calculate the Change in Variance

DALHOUSIE UNIVERSITY

Observed values ($n = 8$)	Two mean-filled gaps ($n = 10$)
$\bar{x} = 84/8 = 10.5$	$\bar{x} = 105/10 = 10.5$
$\sum_i (x_i - \bar{x})^2 = 1134$	$1134 + 0 + 0 = 1134$
$s_x^2 = 1134/7 = 162$	$s_x^2 = 1134/9 = 126$
$s_x = \sqrt{162} \approx 12.73$	$s_x = \sqrt{126} \approx 11.22$

The mean remains 10.5 under any missingness mechanism. MCAR concerns whether this observed mean estimates the population mean without selection bias.

SD falls about 11.8% relative to the observed-data SD. The true complete-data SD is unknown.

Fill Gaps in Ordered Measurements

One subject, five equally spaced times. Sort by time within each subject first.

R

```r
series <- c(10, NA, 14, NA, 18)
forward <- tidyr::fill(data.frame(series),
                       series)$series
baseline <- series
baseline[is.na(baseline)] <- series[1]
inside <- approx(seq_along(series), series,
                 xout = seq_along(series))$y
forward   # 10 10 14 14 18
baseline  # 10 10 14 10 18
inside    # 10 12 14 16 18
```

Python

```python
series = pd.Series([10, np.nan, 14, np.nan, 18])
forward = series.ffill()
baseline = series.fillna(series.iloc[0])
inside = series.interpolate(
    method="linear", limit_area="inside")
print(forward.tolist())
# [10.0, 10.0, 14.0, 14.0, 18.0]
print(baseline.tolist())
# [10.0, 10.0, 14.0, 10.0, 18.0]
print(inside.tolist())
# [10.0, 12.0, 14.0, 16.0, 18.0]
```

LOCF repeats the last observed value and BOCF the baseline value. Interpolation needs an observed value on each side of the gap.

Defend a Missing-Data Decision

A variable has two missing measurements. Would you report its observed values, fill the gaps, or seek more information?

- State the question and the available evidence.
- Name your assumption and the limitation it creates.
- Explain how you would record the decision.

New Representation, Same Meaning

With units, variable types, and missingness decisions documented, choose a representation that suits the analysis.

Today, standardization and discretization.

Express Values in Sample-SD Units

$$z_i = \frac{x_i - \bar{x}}{s_x}$$

Use a numeric variable with at least two observed values and $s_x > 0$.

Constant columns need a separate decision; do not divide by zero.

For $2, 4, 4, 5, 7, 9, 12, 41$, $\bar{x} = 10.5$ and $s_x \approx 12.73$.

$z_1 \approx -0.67$, $z_4 \approx -0.43$, $z_8 \approx 2.40$.

The mean becomes 0 and sample SD becomes 1. Shape and ordering remain unchanged.

Use the Sample-SD Convention

R

```r
x <- c(2, 4, 4, 5, 7, 9, 12, 41)
stopifnot(length(x) > 1, sd(x) > 0)
z <- (x - mean(x)) / sd(x)
z[1] # -0.6678231
mean(z) # approximately 0
sd(z) # 1
as.vector(scale(x))[1] # -0.6678231
```

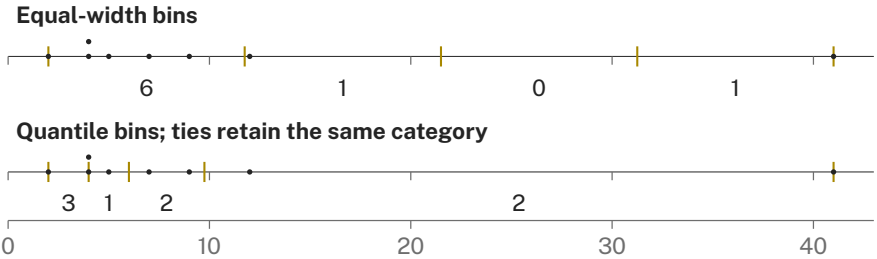
Python

```python
x = pd.Series(
    [2, 4, 4, 5, 7, 9, 12, 41], dtype=float)
assert len(x) > 1 and x.std(ddof=1) > 0
z = (x - x.mean()) / x.std(ddof=1)
print(z.iloc[0]) # -0.6678230711206282
print(z.mean()) # approximately 0
print(z.std(ddof=1)) # 1.0
# Both examples use the sample SD.
```

R's `sd()` and `scale()` and pandas `.std()` use divisor $n - 1$. NumPy's `np.std` and scikit-learn's StandardScaler use n. The Python code states `ddof=1` explicitly.

Equal Widths or Quantile Boundaries?

Equal-width bins split the numeric range into k intervals. Quantile bins aim for similar counts, but ties can prevent equal counts.



Numbers beneath intervals are counts; both axes share the value scale.

Stacked marks at 4 represent the two equal observations.

Save the Boundaries

R

```r
x <- c(2, 4, 4, 5, 7, 9, 12, 41)
k <- 4
width_edges <- seq(min(x), max(x),
                   length.out = k + 1)
quantile_edges <- quantile(
  x, probs = seq(0, 1, 1/k))
table(cut(x, width_edges,
          include.lowest = TRUE))
# 6, 1, 0, 1
table(cut(x, quantile_edges,
          include.lowest = TRUE))
# 3, 1, 2, 2
```

Python

```python
x = pd.Series([2, 4, 4, 5, 7, 9, 12, 41])
k = 4
width_edges = np.linspace(
    x.min(), x.max(), k + 1)
quantile_edges = x.quantile(
    np.linspace(0, 1, k + 1))
print(pd.cut(
    x, width_edges, include_lowest=True
).value_counts(sort=False))
# 6, 1, 0, 1
print(pd.cut(
    x, quantile_edges, include_lowest=True
).value_counts(sort=False))
# 3, 1, 2, 2
```

Include the minimum explicitly and reuse saved edges for comparable categories. If quantile boundaries repeat, use fewer bins and document the choice.

Age Non-Disclosure

A survey records age and consumption. Respondents under 21 are less likely to disclose age, even at the same consumption level. No other variables are recorded.

- What does this suggest about the missingness mechanism?
- Could age categories recover ages that were never reported?
- What can a separate “age not disclosed” flag tell us?

Keep unknown ages unknown. Without evidence, do not label a nonrespondent “under 21.”

What to Take Away

- Define the observational unit before reshaping.
- Use the data dictionary to interpret types, units, and missing codes.
- Report missing counts and justify any deletion or imputation.
- Standardization changes scale; binning reduces detail.
- Keep raw data and record every transformation.

References

[1] R. J. A. Little and D. B. Rubin. *Statistical Analysis with Missing Data*. 3rd ed. Wiley, 2019.

[2] D. B. Rubin. “Inference and missing data”. In: *Biometrika* 63.3 (1976), pp. 581–592.

[3] H. Wickham. “Tidy data”. In: *Journal of Statistical Software* 59.10 (2014), pp. 1–23.